Project Activity 3: IPv4/IPv6 Address Application

Course DEVASC, section 4ITE
UST College of Information and Computing Sciences
John Constantine C. Acuesta
Luis Gabriel C. Araneta
Emanuel R. Ceniza
November 2026

1. Social Coding Selection

We chose Option 2. The activity asks for a prototype that shows public IP addressing to network technicians. Option 2 is a new application, so we can shape the page around the address, the ISP, the ASN, and the country code. Option 1 would add features to a directions script. This class needs the address story.

Social coding means the work is shared in Git. Each change reaches main through a pull request with one reviewer. The repository is https://github.com/ichigokekiii/sys-ad. Section 4 numbers those pull requests 1, 2, 3, and 4.

Address Card is the page for the demo. It is one Flask page. The map fills the screen, and the address card is a modal over that map. The same page is what we open on a phone and on a laptop.

The page has four features.

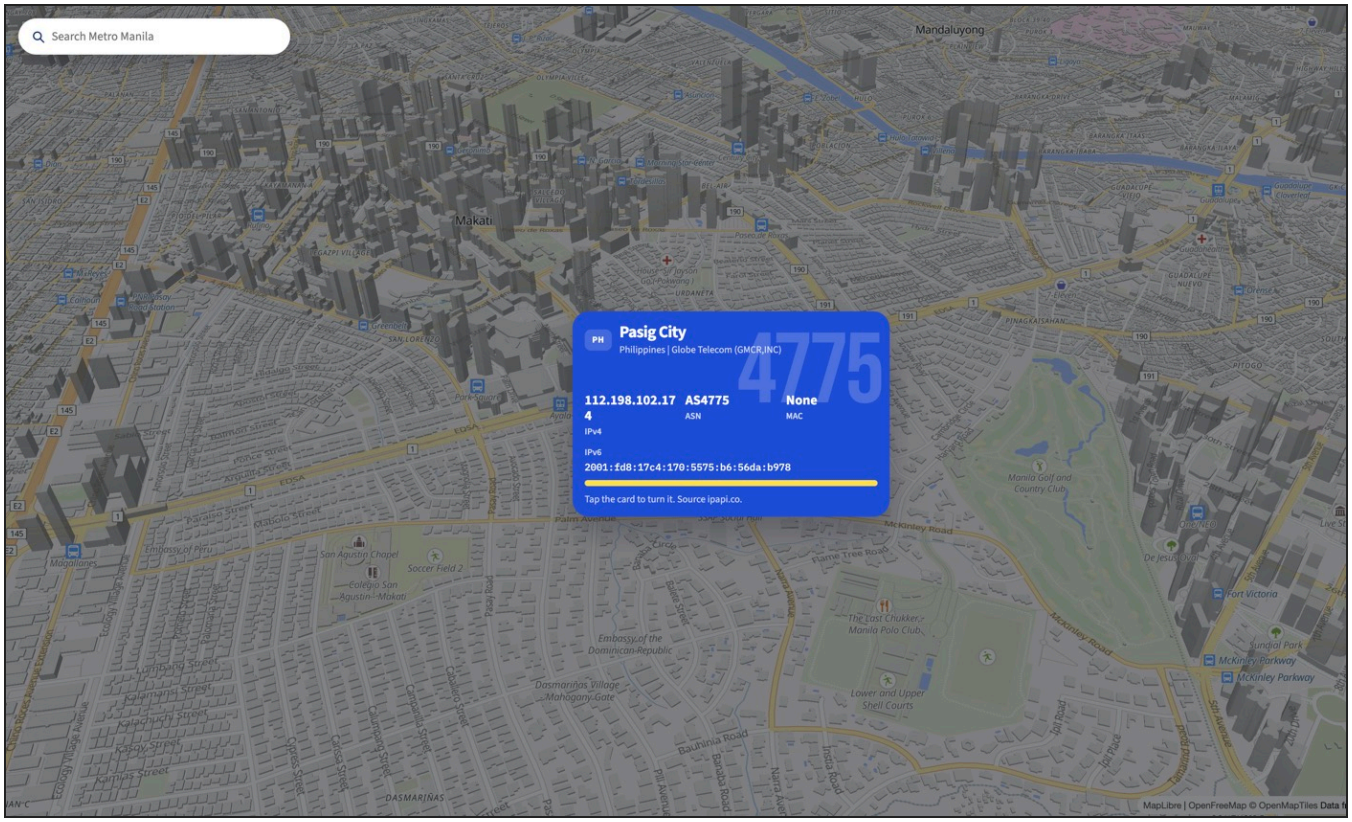


Figure 1. The live page at https://address-card.onrender.com/ on 3 November 2026. The search box is at the top left. The card is open over the map.

- A map that fills the screen. MapLibre draws it with the OpenFreeMap Liberty style. The camera pitches, so buildings can stand up. The tiles are OpenFreeMap, not Mapbox. The

visitor pin is orange. A click on that pin opens the card. A search box at the top left looks up places in Metro Manila, and a chosen place gets a blue pin. The map needs no API key.

- A centered address card. A gray layer covers the map while the card is open. The X sits at the top-right corner of the screen, outside the card. The front lists IPv4, the ASN, and MAC. MAC reads "None", because a website cannot see a device MAC address. IPv6 is its own row. The back holds the roast, the coordinates, the API name, and the one ISP question. The back scrolls inside the card.
- An IPv6 line on the front. The browser fetches `https://api6.ipify.org?format=json`. The line reads "Checking IPv6" until the fetch returns. Failure says "No IPv6 yet".
- One roast and one ISP question. Both sit on the back of the card. The roast is one line about the ISP or the country.

On a laptop and on a phone, the visitor clicks the pin. The card opens in the center. A click on the card flips it, except a click on a quiz choice or on Check. The X at the corner of the screen closes it. The search box opens a place in Metro Manila. That place has no public IP of its own, so the network rows stay this connection's. The place address is one line on the front, and the full line is on the back.

The phone uses that same centered card.

When the visitor asks for reduced motion, or when scripting is off, the card stays flat. Both faces are already in the HTML.

`fun.py` still has `map_embed` for the tests. The page does not show that iframe.

The ISP question has three choices when the ISP is known. The answer is not written into the HTML. The same ISP always rebuilds the same three choices. Grading needs no session. Address Card has no database. The lookup providers and the map need no API key.

The server fills the IPv4 fields in a fixed order. It calls ipwho.is first. If that call fails, it calls ipapi.co. Each call times out after 4 seconds. Any error becomes a notice. The route still returns status 200. Sometimes the visitor's IP is already known and both providers still fail. The page keeps that IP. The notice says that city and ISP could not be loaded. If no IP is known yet, ipify supplies the IPv4 address only. The card records the source as ipwho.is, ipapi.co, ipify, or unavailable.

The server sees the forwarded address and uses it for those IPv4 fields. The browser asks `https://api6.ipify.org?format=json` for the IPv6 line. IPv6 is not part of the server result. The line reads "Checking IPv6" until the fetch returns. A failed fetch sets the line to "No IPv6 yet".

`LOOKUP_MODE=forwarded` reads the first `X-Forwarded-For` entry and looks that address up. A missing header uses `request.remote_addr`. `LOOKUP_MODE=direct` ignores the

forwarding header. In direct mode, ipwho.is reports the server's public address. That address is the home public IP. If `LOOKUP_MODE` is unset, a set `RENDER` variable selects forwarded mode. With `RENDER` unset, the mode is direct. The LAN backup can rely on that default. A machine outside Render already uses direct mode when `LOOKUP_MODE` is unset.

Two notice sentences are fixed. When the lookup is only partly successful, the page says "The location lookup is taking a break. The address below is still yours." When no public address comes back, the page says "We could not find a public address right now. Refresh in a moment."

The layout sample, used while the page was designed, is the documentation address `203.0.113.42`. The sample ISP is PLDT, the ASN is AS9299, the country code is PH, and the city is Manila. The live page replaces those fields with the lookup.

A right answer shows the word Correct in blue, with confetti. The words then fade, and the map moves to the visitor pin. A wrong answer shows the word Wrong in blue, then "This network is " and the ISP name, and the word shakes. Those words sit on the map, not on the card. The grade stored by the server is still "Correct." or "Close. This network is " plus the ISP name. When the ISP is unknown, the page shows no form. The prompt says the provider did not come back.

The activity deliverables are these.

- Address Card, the Flask page, with `app.py`, `ipinfo.py`, `fun.py`, `places.py`, `templates/index.html`, `static/map.js`, `static/card.js`, and `static/search.js`.
- This paper.
- The GitHub repository at https://github.com/ichigokekiii/sys-ad. The branches in section 4 are pull requests 1, 2, 3, and 4.
- The Render page at https://address-card.onrender.com/, plus a LAN backup on John's PC.
- A talk of 10 to 15 minutes, with a live demo on a phone and on a laptop.

The demo has these objectives.

- Show the visitor's public IPv4 address, ISP, ASN, country code, and city on one page.
- Show an IPv6 address, or the words "No IPv6 yet".
- Show the map pin on the lookup coordinates, and say that the pin is often the ISP's equipment.
- Show one roast about the network and one ISP question.
- Stay on the page when a provider fails, and show a notice instead of an error page.
- Reach main only through a reviewed pull request, then demo that result on a phone and on a laptop.

2. Team member roles and skillsets

Roles below are the jobs we assigned.

Team member	Role	Assigned work	Skillset
John Constantine C. Acuesta	Scrum leader and PM	UI and pull request reviews	Project Manager
Luis Gabriel C. Araneta	Recorder and QA	pytest and README review	Scribe
Emanuel R. Ceniza	Developer	API layer and lookup	Developer

John creates the GitHub repository and reviews the pull requests named in section 4. He also builds the page. Luis keeps the task list, writes the pytest tests, and reviews the README. Emanuel builds the API layer and the lookup.

3. Strategy

We will track tasks in a Google Sheet. Luis keeps the sheet, because he is the recorder. The repo copy of that list is `tasks.csv`. We will export the sheet to `tasks.csv` so the list can sit next to the code. This paper does not paste sheet rows. The team fills the rows as the sprints run.

The build is three short sprints. Each sprint ends with something we can open.

Sprint 1 is the lookup. Emanuel will build it on the branch `emanuel/api`. The sprint is done when a request returns a card result. When a provider does not answer, the page shows a notice and stays up.

Sprint 2 is the page. John will build it on the branch `john/ui`. The sprint is done when a phone and a laptop can both show the card, the map, and the ISP question.

Sprint 3 is the tests, the docs, and the demo. Luis will add pytest on the branch `luis/tests` and review the README. John will add this paper on the branch `john/paper`. The sprint is done when three things are true. The tests call the helper functions. The paper is in the repo. We have rehearsed the phone demo and the laptop demo.

We will build in this order.

- First, a lookup that still shows the address when a provider fails.
- Second, a page that works on a phone and on a laptop.
- Third, tests, the README, this paper, and the live demo.

The backlog in section 6 waits until those three are done. A form for an address a technician types, a header lesson, and a classmate compare are later activity work. They are outside these

sprints. We also skipped two ideas that would make the demo risky. Browser location needs more JavaScript than this page can carry. A race among the three providers can fail in the middle of a live demo.

We kept these four features. They can be shown on a phone and on a laptop. They can be built before the demo. They need no API key. The page still renders when one provider is down.

Check-ins. We met in a group chat before each sprint. John named the next piece of the page. Emanuel reported the lookup. Luis wrote the notes and updated the task list. A check-in was short. It covered what was done, what was blocked, and what the next person would open.

4. Using GitHub

The repository is https://github.com/ichigokekiii/sys-ad. `main` is already on that remote. Emanuel's GitHub username is Leunewashere. Luis's GitHub username is lg-araneta. The pull requests are 1, 2, 3, and 4.

John invites lg-araneta and Leunewashere. Luis and Emanuel clone the repository that already exists.

```
git clone https://github.com/ichigokekiii/sys-ad.git
```

Four branches will exist. Each pull request has one reviewer. The author of a branch does not approve that same branch. John is the reviewer on `emanuel/api` and on `luis/tests`. Each of those pull requests still has one reviewer.

Branch	Author	Reviewer	Pull request
emanuel/api	Emanuel R. Ceniza	John Constantine C. Acuesta	1
john/ui	John Constantine C. Acuesta	Luis Gabriel C. Araneta	2
luis/tests	Luis Gabriel C. Araneta	John Constantine C. Acuesta	3
john/paper	John Constantine C. Acuesta	Emanuel R. Ceniza	4

Luis reviews the README as QA. The table still names the one person who approves each pull request.

The branch flow is the same for all four. This is the lookup branch.

```
git checkout main
git pull
git checkout -b emanuel/api
```

After the lookup files are ready, Emanuel commits and pushes.

```
git add .
git commit -m "Add the lookup fallback"
git push -u origin emanuel/api
```

He then opens a pull request on GitHub from `emanuel/api` into `main`. The pull request number is 1. John is the reviewer. John approves the pull request, or he asks for a change on the same branch. Emanuel pushes any requested change to `emanuel/api`. Merge to `main` waits until John approves.

The other branches use the same commands. Only the branch name, the commit message, and the reviewer change.

- `john/ui` uses a message in the shape "Add the Address Card page". Luis reviews it.
- `luis/tests` uses a message in the shape "Add pytest for the lookup helpers". John reviews it.
- `john/paper` uses a message in the shape "Add the Activity 3 paper". Emanuel reviews it.

After a merge, the author updates local `main`.

```
git checkout main
git pull
```

If git reports a conflict, the author fixes it on the feature branch and pushes that branch again. The merge to `main` still waits for the assigned reviewer.

5. Live deployed link

The live page is https://address-card.onrender.com/. The free instance sleeps, so open it a few minutes before the demo.

6. Backlog

These items are real later work. They are not part of the demo. The same list is on GitHub for Project Activity 4, at https://github.com/ichigokekiii/sys-ad/issues.

- Look up an address a technician types. https://github.com/ichigokekiii/sys-ad/issues/1
- Show the raw JSON for class discussion. https://github.com/ichigokekiii/sys-ad/issues/2

- Compare two addresses. https://github.com/ichigokekiii/sys-ad/issues/3
- Save a lookup to a CSV file. https://github.com/ichigokekiii/sys-ad/issues/4
- Add more roast lines written by the class. https://github.com/ichigokekiii/sys-ad/issues/5
- Look up IPv6 on the server when the host itself has IPv6. https://github.com/ichigokekiii/sys-ad/issues/6

The header lesson and the classmate compare sit with that later work. The plan calls them Activity 4 work. Browser location and a latency race are not on this list. We dropped them because they make a live demo fragile.

7. Team activities and reflection

Planned activities.

- Luis keeps the task list and the notes.
- We run the three sprints in section 3.
- Each branch in section 4 is one pull request with the reviewer named there.
- We rehearse the talk on the laptop and on the phone before class.
- Check-ins. We met in a group chat before each sprint. John named the next piece of the page. Emanuel reported the lookup. Luis wrote the notes and updated the task list.

What worked well.

The split in section 2 held. John ran the page and the demo plan. Emanuel owned the lookup and the fallback order. Luis kept the notes, the task list, and the test review. The page still opens when one provider fails, so a bad call does not end the demo.

Team problems and fixes.

The first copy of the app sat in a nested folder, so GitHub would not take that folder as the project. John moved the Flask app to the repository root and removed the extra folder. After that, `main` on https://github.com/ichigokekiii/sys-ad matched the page we demo.

Technical problems and fixes.

A website cannot read a device MAC address. The card says "None" instead of a made-up address. The server also cannot see the visitor's IPv6 address on this host, so the browser asks api6.ipify.org and the row says "No IPv6 yet" when that fetch fails. The lookup tries ipwho.is, then ipapi.co, then a notice or ipify. The route still returns status 200.

Accountability.

John delivered the page, the paper, and the reviews named for the project manager. Emanuel delivered the lookup path in `ipinfo.py` and `fun.py`. Luis delivered the scribe work, the task

notes, and the pytest review. The Activity 4 items stayed on the GitHub issue list instead of slipping into the demo.

Decision-making.

We kept the pitched map, the orange visitor pin, the blue place pin, the IPv6 row, and one roast with one ISP question. A searched place in Metro Manila has no public IP of its own, so the card does not invent one. The typed lookup, the header lesson, and the classmate compare stay on the backlog in section 6.

Team dynamics and lessons learned.

A short check-in was enough when each person had one job. The useful lesson was to say what the page cannot know. MAC stays "None", a searched place has no public IP, and the free Render instance sleeps until someone opens it.